Docker Architecture

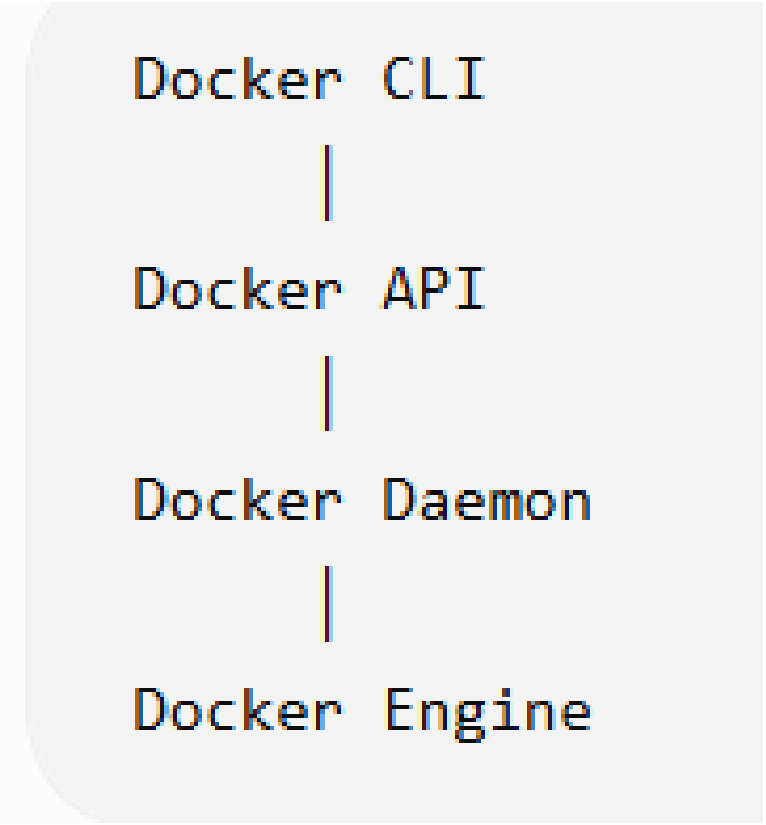
1. Host Operating System

- The **host OS** is the operating system installed on your physical machine.
- Examples:
  - Windows
  - macOS
  - Linux
- Docker components run/interact with this host operating system

2. Docker Engine

Docker Engine consist of

- Docker CLI
- Docker Daemon
- Docker API



- Docker Engine interacts with :
  - Docker Images
  - Docker Containers
  - Docker Registry

3. Docker CLI

- CLI = Command Line Interface
- It is the client through which users interact with Docker
- Users execute Docker commands through CLI
- The Docker CLI communicates with the Docker Daemon.
- The daemon then executes/manages the requested operation
- Example:

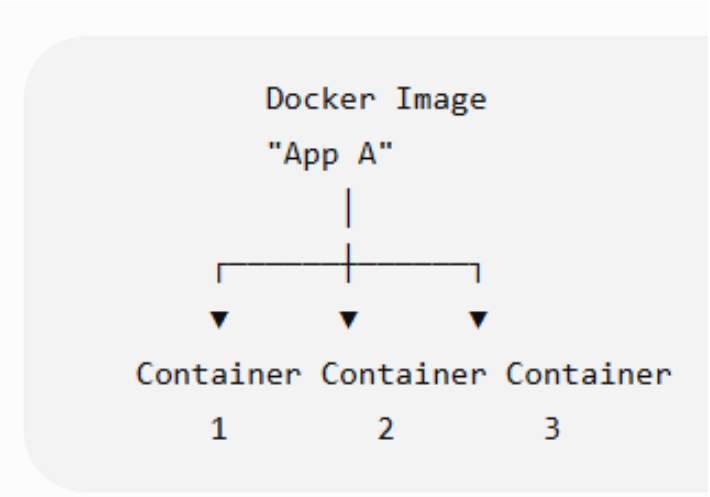
```bash
docker run
docker build
docker pull
docker push
```

4. Docker Daemon

- the **Docker Daemon** runs on the Host Operating System.
- It is responsible for :-
  - Building Docker Images
  - Managing containers
  - Managing Docker objects
- It listens to the commands sent through the Docker CLI
- It also interacts with the Docker API
- It manages Docker:
  - Images
  - Containers
  - Other Docker objects
- Simple idea :-
  - Daemon = the component that actually manages the Docker resources and executes requests.

5. Docker Image

- A Docker image is a template/blueprint for creating containers.
- It contains the instructions/content required to create a container.
- One image can be used to create **multiple containers.**
- Example:



6. Docker Container

- A **Docker Container** is a running instance of a **Docker Image**
- Containers run applications along with their required
  - Libraries
  - Binaries
- **Multiple containers** can be created from the **same image.**
- **Docker daemon** manages the containers.
- containers are **isolated environments** for running the applications.

7. Docker Registry

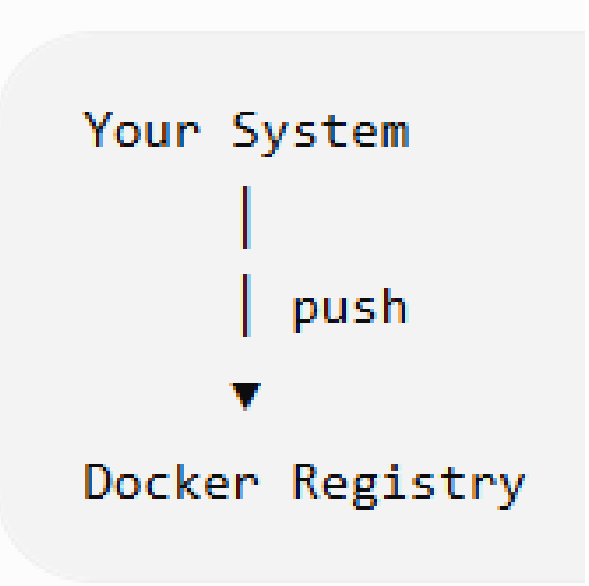
- A **Docker Registry** is a service that stores Docker images.
- Images in a registry can be :
  - **Public**
  - **Private**
- Public images can be accessed by users who have access to the registry.
- Private images are restricted to authorized users.
- The registry acts a: **Central repository for Docker Images.**

8. Docker pull

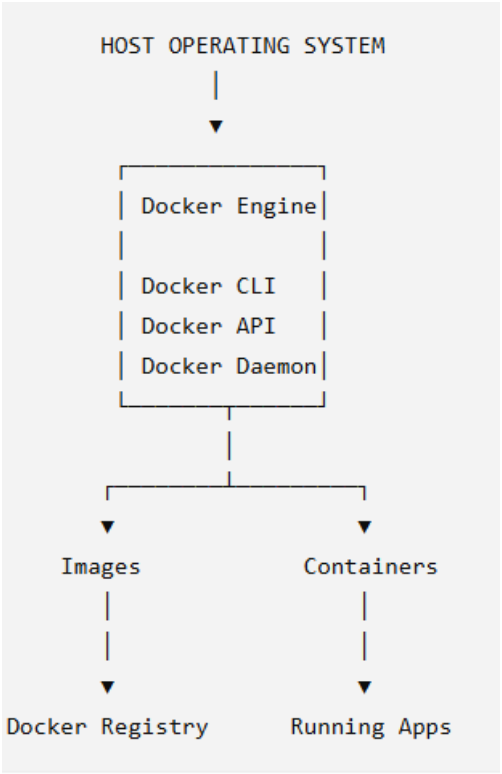
- Pull means getting an image **from the Docker registry to your system.**
- You pull an image when you need to obtain it locally
- After obtaining the image, we can run it to create a container.

9. Docker Push

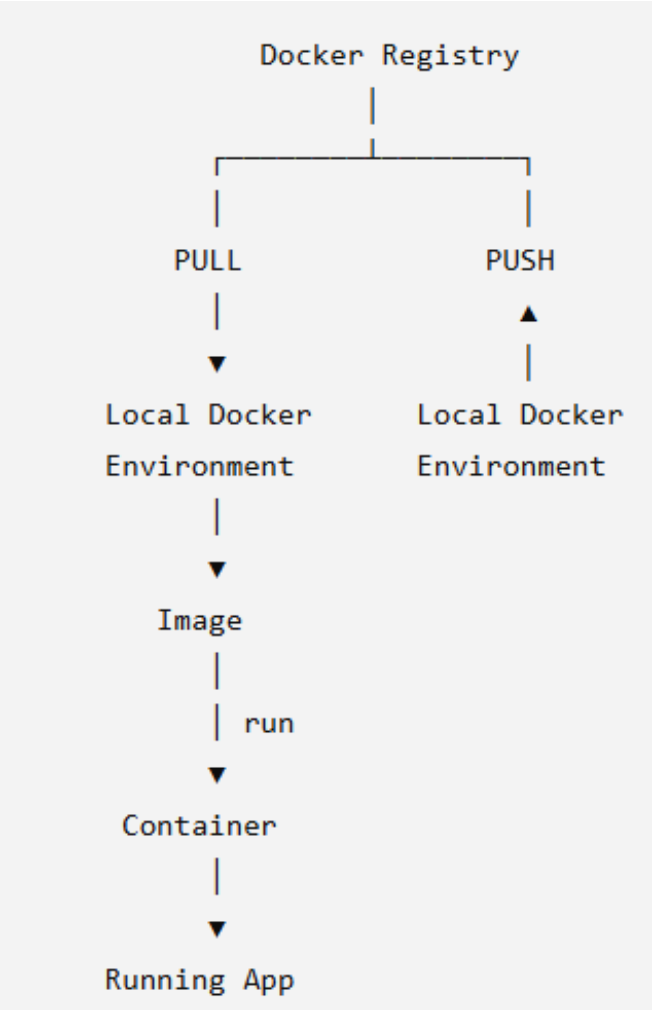
- Push means sending an image **from your system to the Docker Registry.**
- we push an image when we want to store/share/distribute it through the registry
- This allows the image to be accessed from other systems according to the registry's access permissions.



10. Overall Docker Architecture



11. Registry Interaction



Terms

Term	Simple meaning
Host OS	OS of your machine
Docker CLI	Interface where you give Docker commands
Docker API	Communication interface used by Docker components
Docker Daemon	Manages Docker objects and performs operations
Docker Engine	Docker's core engine/components
Image	Blueprint/template
Container	Running instance of an image
Registry	Repository for Docker images
Pull	Registry → Local system
Push	Local system → Registry

One-Line interview Explanation

- **Docker** uses client-server architecture where the **Docker CLI** communicates with the **Docker Daemon** through the **Docker API**.
- The daemon manages images, containers and other Docker objects.
- While **docker registries** store and distribute **Docker images** through operations such as **pull** and **push**